

Square Order Injection Sprint

Date: 2026-07-08

Status: Implemented (MVP — one-way injection)

Summary

Paid Open Order vendor orders can now be routed into Square as external/prepaid pickup orders when a vendor is explicitly configured for Square routing, has a healthy Square connection, a published Square-imported menu, and complete `ProviderEntityMapping` coverage for line items and modifiers.

Customer payment remains on Stripe. Square receives an `EXTERNAL` payment record so the order appears paid in Square POS without collecting from the customer. Open Order remains merchant of record.

Files changed

Area	Files
Schema / migration	<code>prisma/schema.prisma</code> , <code>prisma/migrations/20260708140000_square_order_injection/</code>
Routing entry	<code>src/services/routing.service.ts</code>
Square order service	<code>src/services/square-order.service.ts</code>
Mapper + types	<code>src/lib/integrations/square/square-order-mapper.ts</code> , <code>square-order.types.ts</code>
Readiness gating	<code>src/lib/integrations/square/square-order-routing-readiness.ts</code>
Square API client	<code>src/lib/integrations/square/square-api.client.ts</code>
Env flag	<code>src/lib/env.ts</code> (<code>SQUARE_ROUTING_LIVE</code>)
Vendor routing mode	<code>src/lib/vendor-order-routing-mode.ts</code>
Admin enable toggle	<code>src/services/admin-vendor-rescue.service.ts</code> , <code>src/actions/admin-vendor.actions.ts</code> , <code>AdminVendorOrderRoutingSection.tsx</code>

Admin order observability	<code>src/lib/admin-order-detail-query.ts</code> , <code>admin-order-detail-ui.ts</code> , <code>AdminVendorOrderOperationalPanel.tsx</code> , <code>AdminDeliverectDiagnosticsPanel.tsx</code>
Retry availability	<code>src/lib/routing-availability.ts</code>
Tests	<code>square-order-mapper.test.ts</code> , <code>square-order-routing-readiness.test.ts</code> , <code>square-order.service.test.ts</code> , <code>routing.service.test.ts</code> , <code>vendor-order-routing-mode.test.ts</code>

Part 1 — Routing mode behavior

Supported modes

- `manual_dashboard` — unchanged; paid orders stay in OO vendor queue
- `deliverect` — unchanged; Deliverect submit path
- `square` — new operational path when explicitly enabled

Square injection gates (all required)

1. `Vendor.orderRoutingMode === "square"`
2. `Vendor.squareOrderRoutingEnabled === true` (admin toggle; **not** auto-enabled on menu publish)
3. Square connection healthy (OAuth token valid, integration active)
4. Selected Square location
(`VendorIntegrationConnection.externalLocationId`)
5. Published live menu with `sourcePayloadKind === "square_catalog_v1"`
6. Per-order: every line item and modifier has an active `ProviderEntityMapping`
7. `SQUARE_ROUTING_LIVE=true` for live API calls (otherwise payload is built and stored without send)

Hook point: `submitVendorOrder()` in `routing.service.ts` , called post-Stripe payment success.

Part 2 — Order mapping

`mapVendorOrderToSquareCreateOrder()` maps:

OO entity	Mapping key	Square field
-----------	-------------	--------------

<code>MenuItem.deliverectProductId</code> (<code>sq:prod:*</code>)	<code>ProviderEntityMapping</code> (<code>menu_item</code>)	<code>line_items[].catalog_objec</code>
<code>ModifierOption.deliverectModifierId</code> (<code>sq:modopt:*</code>)	<code>ProviderEntityMapping</code> (<code>modifier_option</code>)	<code>line_items[].modifiers[].c</code>

Included in payload:

- Item quantity
- Selected modifiers
- Order notes + line special instructions
- Pickup fulfillment (`type: PICKUP` , `schedule_type: ASAP`)
- Pickup code in fulfillment note
- `source.name: "Open Order"`
- `location_id` from connection
- `reference_id` = vendor order id
- Idempotency key: `oo:sq:order:{vendorOrderId}`

Missing mappings

- Routing blocked before API send
- `VendorOrder.routingStatus = failed`
- `routing_failure` issue created (HIGH)
- No silent item/modifier drops

Part 3 — Square order creation

Flow in `submitVendorOrderToSquare()` :

1. Load vendor order + assert vendor-level readiness
2. Map payload via `ProviderEntityMapping`
3. `CreateOrder` using **vendor OAuth token** (`ensureSquareAccessToken`)
4. `CreatePayment` with `source_id: "EXTERNAL"` and idempotency `oo:sq:pay:{vendorOrderId}`

Persisted audit fields (`VendorOrder`)

Field	Purpose
<code>squareOrderId</code>	External Square order id
<code>squareSubmittedAt</code>	Successful submit timestamp

<code>squareAttempts</code>	Incremented on failure
<code>squareLastError</code>	Last error message
<code>lastSquarePayload</code>	Request audit (create order + payment)
<code>lastSquareResponse</code>	API responses
<code>routingStatus</code>	<code>sent</code> on success, <code>failed</code> on hard failure

`ProviderEntityMapping` upserted for `vendor_order` → Square order id.

Idempotent: if `routingStatus=sent` and `squareOrderId` exists, submit is skipped.

Part 4 — Payment / prepaid treatment

Square does **not** charge the customer.

After `CreateOrder`, Open Order records payment via Square Payments API:

```
{
  "source_id": "EXTERNAL",
  "external_details": { "type": "OTHER", "source": "Open Order" },
  "amount_money": { "amount": <Square order total>, "currency": "USD" },
  "autocomplete": true
}
```

This marks the Square order as paid for POS/kitchen visibility. Stripe checkout, payouts, and pod payouts are unchanged.

Known limitation: External payment amount uses Square-calculated `order.total_money`, which may differ slightly from the OO/Stripe charge due to catalog pricing or tax rounding.

Part 5 — Status handling (MVP)

One-way injection only:

- Success → `routingStatus = sent`
 - Fulfillment remains OO-controlled (manual dashboard / vendor kitchen UI)
 - No Square status webhooks in MVP
-

Part 6 — Fallback behavior

On failure:

- Paid OO order remains visible in admin/vendor tools
 - `routing_failure` issue surfaced
 - Admin **Retry routing** available when `SQUARE_ROUTING_LIVE=true` (or Deliverect live)
 - Manual recovery unchanged
 - No automatic refund
-

Part 7 — Admin / vendor observability

Admin vendor rescue (`/admin/vendors/{id}`)

- Square order routing enable/disable toggle
- Prerequisites checklist via `loadSquareOrderRoutingReadiness()`

Admin order detail

- Operational panel: Square provider label, Square order id, routing error
- Collapsible **Square routing details**: status, submit time, mapping issues, raw payload/response
- Existing retry routing + manual recovery actions

Vendor dashboard

- Routing status labels via existing `vendorRoutingStatusLabel()` (no Square-specific POS lock like Deliverect)
-

Part 8 — Tests / QA

#	Scenario	Test file
1	Square routing disabled unless explicitly enabled	<code>routing.service.test.ts</code> , <code>square-order-routing-readiness.test.ts</code>
2	Requires healthy connection	<code>square-order-routing-readiness.test.ts</code>
3	Requires active location	<code>square-order-routing-readiness.test.ts</code>
4	Requires Square-imported published menu	<code>square-order-routing-readiness.test.ts</code>

5	Item → Square variation ID	<code>square-order-mapper.test.ts</code>
6	Modifier → Square modifier ID	<code>square-order-mapper.test.ts</code>
7	Missing item mapping blocks routing	<code>square-order-mapper.test.ts</code>
8	Missing modifier mapping blocks routing	<code>square-order-mapper.test.ts</code>
9	Pickup fulfillment in payload	<code>square-order-mapper.test.ts</code>
10	Vendor OAuth token used	<code>square-order.service.test.ts</code>
11	Idempotency prevents duplicate orders	<code>square-order.service.test.ts</code>
12	API failure creates routing failure	<code>square-order.service.test.ts</code>
13–14	Deliverect/manual unchanged	<code>routing.service.test.ts</code>
15	Build passes	<code>npm run build</code>

74 Square/routing-related unit tests passing.

Manual QA checklist

1. Connect Square sandbox vendor, import + publish menu
2. Admin: set routing mode `square`, enable Square order routing
3. Place test order with Stripe test card
4. Verify `VendorOrder.squareOrderId` populated in admin
5. Verify order visible in Square sandbox dashboard
6. Disable `SQUARE_ROUTING_LIVE` → confirm payload-only skip (no API call)
7. Remove a mapping → confirm `routing_failure` issue, no partial send

Environment

Variable	Purpose
<code>SQUARE_ROUTING_LIVE=true</code>	Enable live Square CreateOrder + EXTERNAL payment
Square OAuth vars	Existing Square connection (unchanged)

Known limitations

1. One-way injection — no Square webhook reconciliation
 2. Square order total may differ from Stripe charge for EXTERNAL payment amount
 3. `SQUARE_ROUTING_LIVE` must be set per environment for live sends
 4. Square order routing must be **manually enabled** by admin after menu publish
 5. Deliverect and manual routing paths untouched
-

Rollout steps

1. Apply migration: `npx prisma migrate deploy`
2. Set `SQUARE_ROUTING_LIVE=true` in staging/production when ready
3. Per vendor: publish Square menu → set routing mode `square` → admin enable Square order routing